AI-Driven Financial Fraud Detection, Risk Scoring & Explainable Decision Support

Complex Computing Problem (CCP) Report Data Science & Visualization Lab, BS Artificial Intelligence (7th Semester), Iqra University

Author: Aleem Ul Hassan

Executive Summary

This report accompanies the implementation notebook (`notebooks/fraud_detection_ccp.ipynb`) and covers the conceptual and analytical tasks of the CCP: business understanding (Task 1), scalability and big-data architecture (Task 10), a literature review of recent research (Task 11), and a critical evaluation (Task 12). The implementation builds a complete fraud-analytics pipeline on mobile-money transaction data (PaySim schema): exploratory analysis, behavioral feature engineering, imbalance handling, six supervised and deep models, explainability with SHAP and LIME, a dynamic risk-scoring engine, unsupervised anomaly discovery, and a four-view business-intelligence dashboard. Key implementation results are summarized at the end of this report with figures generated by the pipeline.

The central business reality that shapes every technical decision is **extreme class imbalance**: genuine fraud is far below one percent of transactions, so accuracy is meaningless and the system is instead optimized for recall and precision-recall trade-off, with explainability built in so every alert can be justified to an analyst, a customer, and a regulator.

Task 1 - Business Understanding and Problem Analysis

1.1 The business context

A multinational digital bank processes millions of transactions daily across many countries and payment rails. Traditional rule-based engines (for example, "flag any transfer above a fixed threshold") are easy to understand but brittle: fraudsters learn the thresholds and structure transactions to stay just underneath them, while the rules generate large volumes of false alarms that overwhelm investigators. The goal is a learning system that adapts to new fraud patterns, scores risk continuously, and explains its decisions.

1.2 Fraud types

- **Account takeover (ATO):** an attacker gains control of a legitimate account through phishing or credential stuffing, then drains it. In mobile money this shows up as a `TRANSFER` out followed by a `CASH_OUT`, often emptying the balance.

- **Transaction / payment fraud:** unauthorized card-not-present or wallet payments using stolen credentials.
- **Identity theft / synthetic identity:** new accounts opened with fabricated or stolen identities to launder funds or take loans.
- **Money laundering:** layering funds through chains of transfers and cash-outs to obscure their origin; this is inherently a network/graph problem.
- **Authorized push payment (APP) / social-engineering fraud:** the genuine customer is tricked into authorizing a payment to a fraudster, which is hard to catch because the credentials and device are legitimate.

In the PaySim data, fraud appears only in `TRANSFER` and `CASH_OUT`, mirroring the ATO-to-cash-out pattern, which is why our engineered balance-error features are so predictive.

1.3 Core challenges

- **Severe class imbalance** (fraud well under 1%), which biases naive models toward the majority class.
- **Concept drift:** fraud tactics change continuously, so a model trained on last quarter's fraud degrades over time.
- **Adversarial adaptation:** unlike most ML problems, the data-generating process actively tries to defeat the model.
- **Real-time constraints:** scoring must complete within tens of milliseconds so as not to delay legitimate payments.
- **Label scarcity and delay:** confirmed-fraud labels arrive late (after chargebacks/investigations), so supervised data is always incomplete.

1.4 False positives vs false negatives

Fraud detection is a problem of **asymmetric, conflicting costs**:

- A **false negative** (missed fraud) means direct financial loss, regulatory exposure, and reputational damage.
- A **false positive** (a legitimate transaction blocked) means customer friction, abandoned purchases, call-center load, and churn. A single high-value customer wrongly declined at a critical moment can cost more in lifetime value than several small frauds.

Because the classes are so imbalanced, even a very low false-positive *rate* produces a large *absolute* number of false alarms. This is why the project optimizes the **precision-recall trade-off** and exposes a tunable **risk-score threshold** (Task 7) rather than a single hard cut-off: operations can move the Low/Medium/High/Critical boundaries to match their investigator capacity and risk appetite.

1.5 Customer impact

Customers experience fraud controls as friction (step-up authentication, declined payments, frozen accounts) or as harm (undetected fraud draining their funds). A trustworthy system minimizes both by being accurate *and* explainable, so that when a transaction is challenged the customer can be given a clear, fair reason and a fast resolution path.

1.6 Regulatory considerations

- **AML / KYC** (Anti-Money-Laundering, Know-Your-Customer) and the Bank Secrecy Act require monitoring, suspicious-activity reporting, and auditable decisions.
- **PSD2 Strong Customer Authentication (SCA)** in the EU mandates multi-factor authentication and shapes when step-up challenges fire.
- **GDPR Article 22** gives individuals a right not to be subject to solely automated decisions with significant effects, and a right to meaningful information about the logic involved, which is precisely why SHAP/LIME explainability (Task 6) is a compliance requirement, not a nice-to-have.
- **Fair-lending / anti-discrimination law** (for example ECOA in the US) requires that models not produce disparate impact across protected groups, implying ongoing fairness monitoring.
- **PCI-DSS and GLBA** govern the secure handling of payment and customer data that the pipeline ingests.

Task 10 - Scalability and Big Data Considerations

A production deployment for "millions of transactions daily across multiple countries" cannot run in a single notebook on one machine. The pipeline maps onto a standard big-data, streaming-first architecture.

10.1 Reference architecture (Lambda / Kappa)

A **Kappa-style streaming architecture** is the natural fit because fraud scoring must be real-time:

```
Producers (apps, POS, wallets)
        |
   Apache Kafka  ---- topic: raw-transactions ----+
        |                                          |
   Spark Structured Streaming                 (cold path) HDFS / cloud object store
   - feature enrichment (feature store)            |
   - model scoring (low latency)              Batch retraining (Spark MLlib / XGBoost on
Spark)
        |                                          |
   Risk score + decision  <----- model registry --+
        |
   Alert queue (Low/Med/High/Critical)  ->  case-management UI + dashboards
```

10.2 Component roles

- **Apache Kafka** is the ingestion backbone: a durable, partitioned, horizontally scalable log that decouples producers from consumers and absorbs spikes. Transactions land on a topic and are consumed by the scoring service with at-least-once guarantees.

- **Apache Spark (Structured Streaming + MLlib)** provides distributed feature computation and model scoring in micro-batches or continuous mode, and distributed *training* on the full historical dataset that will not fit in memory on one node. Tree ensembles and logistic regression scale through Spark MLlib; XGBoost and LightGBM offer their own distributed/Spark backends.
- **Hadoop / HDFS (or cloud object storage such as S3/GCS)** is the cold-path data lake for the years of historical transactions used in batch retraining and audit.
- **A feature store** (for example Feast) guarantees that the features computed at scoring time match those used at training time (no training-serving skew), and serves precomputed aggregates (such as a customer's rolling transaction velocity) with millisecond latency.
- **Model serving and registry** (MLflow / SageMaker / Vertex) version models, support shadow and canary deployment, and enable instant rollback when a model degrades.

10.3 Cloud deployment

- **AWS:** Kinesis or MSK (Kafka) for ingestion, EMR or Glue (Spark) for processing, SageMaker for training, hosting, and Model Monitor for drift, S3 as the lake.
- **GCP:** Pub/Sub for ingestion, Dataflow for streaming, Vertex AI for training and serving, BigQuery and GCS for analytics and storage.

10.4 Non-functional requirements

- **Latency budget:** end-to-end scoring within roughly 50-100 ms so payment authorization is not delayed.
- **Throughput:** thousands of transactions per second, handled by partitioning Kafka topics and scaling Spark executors horizontally.
- **Drift and monitoring:** continuous tracking of feature distributions, score distributions, and realized fraud rates, with automated retraining triggers.
- **Governance:** every decision logged with its SHAP explanation for audit and dispute resolution.

Task 11 - Research and Literature Review (2023-2026)

Four recent peer-reviewed and preprint works frame where this project sits in the current research landscape. They cluster around three themes that the implementation directly engages: **explainability**, **imbalance handling with ensembles**, and the emerging **graph-based** frontier.

11.1 Explainable AI with ensemble models

Almalki, F., & Masud, M. (2025). *Financial Fraud Detection Using Explainable AI and Stacking Ensemble Methods.* arXiv:2505.10050. This work stacks three gradient-boosting models (XGBoost, LightGBM, CatBoost) and layers explainability on top using SHAP together with LIME, partial-dependence plots, and permutation importance. Evaluated on the IEEE-CIS Fraud Detection dataset (over 590,000 records), it reports about 99% accuracy and an AUC-ROC near 0.99, and argues that high predictive performance and transparency are not in conflict. This validates the design choice in our Task 5 and Task 6 to pair strong boosted-tree models with SHAP and LIME rather than treating accuracy and interpretability as a trade-off.

11.2 Interpretable framework for imbalanced data

Baisholan, N., Dietz, J. E., Gnatyuk, S., Turdalyuly, M., & Matson, E. T. (2025). ***FraudX AI: An Interpretable Machine Learning Framework for Credit Card Fraud Detection on Imbalanced Datasets.*** **Computers, 14(4), 120.** FraudX AI combines Random Forest and XGBoost, averaging their probabilities and optimizing the decision threshold, and uses SHAP for interpretation. On the imbalanced European credit-card dataset it achieves about 95% recall and a 97% AUC-PR while keeping false positives low. Its emphasis on PR-AUC (rather than accuracy) and threshold optimization for an imbalanced setting is exactly the evaluation philosophy adopted in our Tasks 4, 5 and 7.

11.3 Explainable ensembles for mobile money specifically

Vijayanand, D., & Smrithy, G. S. (2025). ***Explainable AI-enhanced ensemble learning for financial fraud detection in mobile money transactions.*** **Journal of Algorithms & Computational Technology (SAGE), DOI 10.1177/18724981241289751.** This is the closest work to our problem domain because it targets **mobile-money** transactions (the same PaySim-style setting), combining ensemble learning with explainable AI. It confirms that the engineered behavioral and balance features, ensemble classifiers, and SHAP-based explanations used here reflect current best practice for the mobile-money fraud problem specifically, not just generic credit-card fraud.

11.4 The graph-based frontier

Cheng, D., Zou, Y., Xiang, S., & Jiang, C. (2024). ***Graph Neural Networks for Financial Fraud Detection: A Review.*** **arXiv:2411.05815.** Surveying over 100 studies, this review shows that Graph Neural Networks capture relational structure (shared devices, counterparties, money-flow chains) that tabular models miss, and that they outperform traditional methods on networked fraud such as money-laundering rings. It is the basis for the "graph neural networks" recommendation in our Task 12 future work: the tabular pipeline here is strong on per-transaction signals but blind to the network structure GNNs exploit.

11.5 Synthesis and research gap

Across these works the consensus is clear: boosted-tree ensembles remain the strongest tabular baseline, PR-AUC and recall are the right metrics under imbalance, and SHAP/LIME explainability is now expected rather than optional. The open frontier is **relational and temporal modeling** (GNNs and dynamic graphs) and **robustness to drift and adversarial adaptation**, which single-snapshot tabular models, including this one, do not yet address.

Task 12 - Critical Evaluation

12.1 Technical limitations

- **Synthetic data realism.** PaySim is a simulator (and where a live mirror is unavailable, this project falls back to a reproducible synthetic generator with the same schema and fraud signal). Both reproduce the *structure* of mobile-money fraud but not the full messiness of real production data, so the absolute metrics here are optimistic relative to a real deployment.
- **Concept drift.** The model is trained on a static snapshot. Fraud evolves, so in production performance decays without scheduled retraining and drift monitoring.
- **Adversarial adaptation.** Fraudsters probe and adapt to deployed models; static evaluation overstates durable performance.
- **Imbalance ceiling.** Even with SMOTE-family resampling, the extreme minority rate limits achievable precision at high recall; synthetic minority samples can also blur the decision boundary.
- **Tabular blind spot.** The pipeline scores each transaction independently and cannot see the network of accounts, devices, and money flows that characterize laundering rings.
- **Manual feature engineering.** The behavioral features (balance errors, draining flags) are hand-crafted; they encode known fraud mechanics but may miss novel patterns that representation-learning approaches would capture.
- **Deep-model substitution.** In this lean build the ANN and Autoencoder use scikit-learn (`MLPClassifier` / `MLPRegressor`) rather than a deep-learning framework. They are faithful neural networks but smaller and less tunable than a TensorFlow/PyTorch implementation would be.

12.2 Business limitations

- **False-positive cost.** High recall comes with false positives that create customer friction and investigation load; the right operating point is a business decision, not a purely technical one.
- **Explainability vs performance.** Although SHAP/LIME mitigate it, the most accurate models are still less transparent than simple rules, which matters for regulatory sign-off.
- **Operational integration.** A model is only useful inside a case-management workflow with analyst feedback loops; that organizational cost is outside the notebook.
- **Regulatory and fairness exposure.** Without protected-attribute data, formal fairness was not measured here; a production system must monitor disparate impact continuously.

12.3 Future enhancements

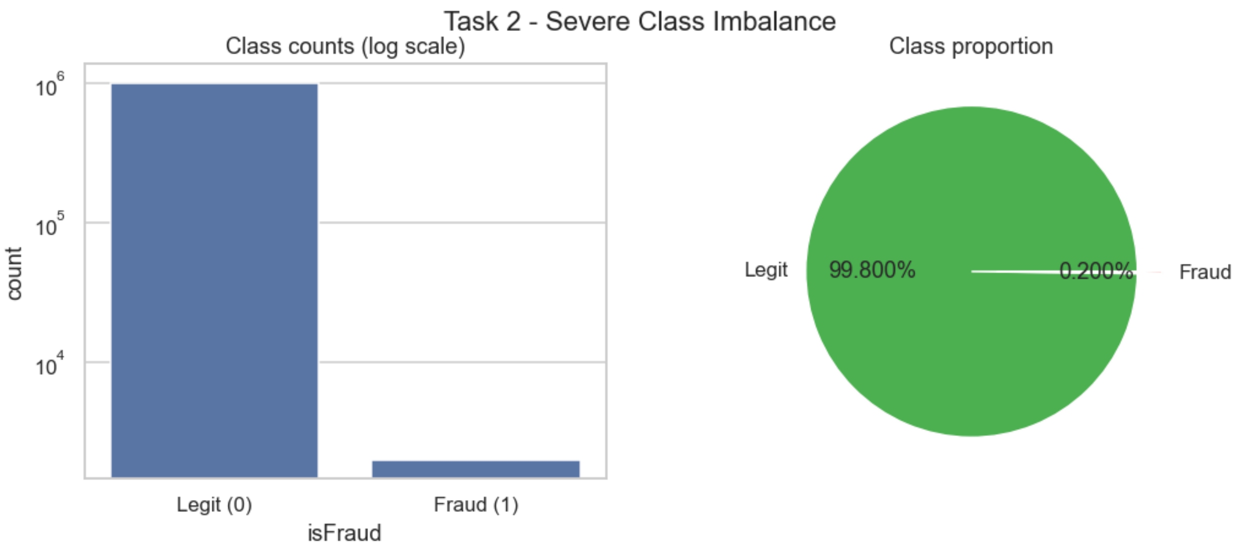
- **Graph Neural Networks** to model the account-device-counterparty network and catch laundering rings (per Cheng et al., 2024).
- **Real-time streaming** deployment on Kafka + Spark with a feature store, as outlined in Task 10.
- **Online / active learning** to incorporate late-arriving fraud labels and adapt to drift continuously.
- **Federated learning** to learn across institutions or regions without sharing raw customer data, improving coverage while respecting privacy.

- **Behavioral biometrics and device intelligence** as additional features for authorized-push-payment and account-takeover fraud.
- **LLM-assisted investigation agents** to summarize an alert, its SHAP drivers, and related transactions into a narrative that accelerates analyst review.

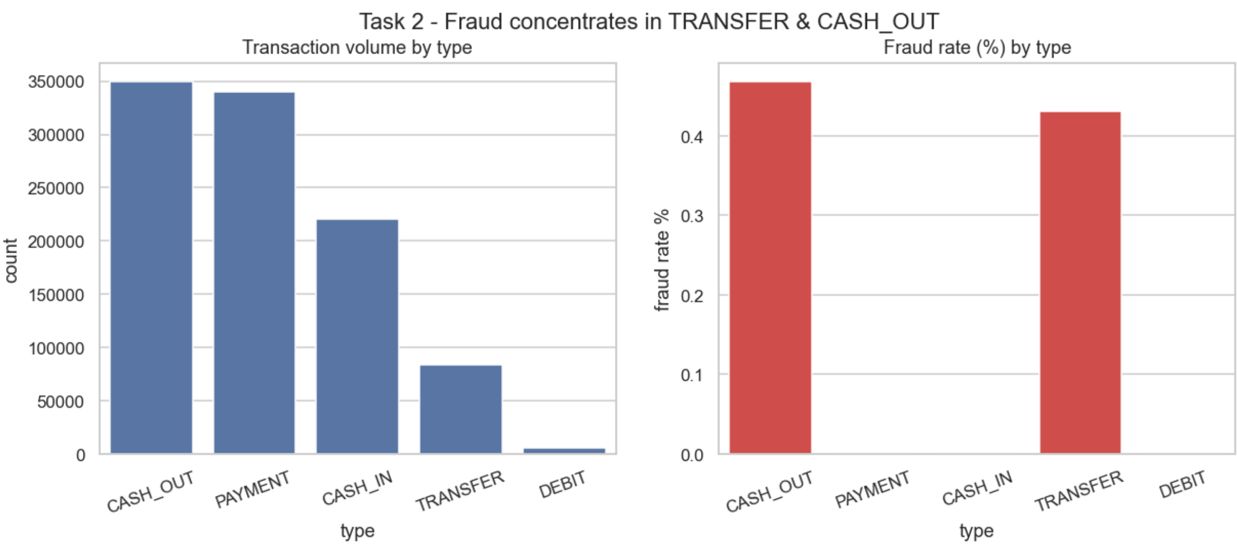
Implementation Summary (Tasks 2-9)

The figures below are generated by the pipeline (`outputs/figures/`). Full code and per-cell outputs are in `notebooks/fraud_detection_ccp.ipynb`.

Exploratory analysis (Task 2)

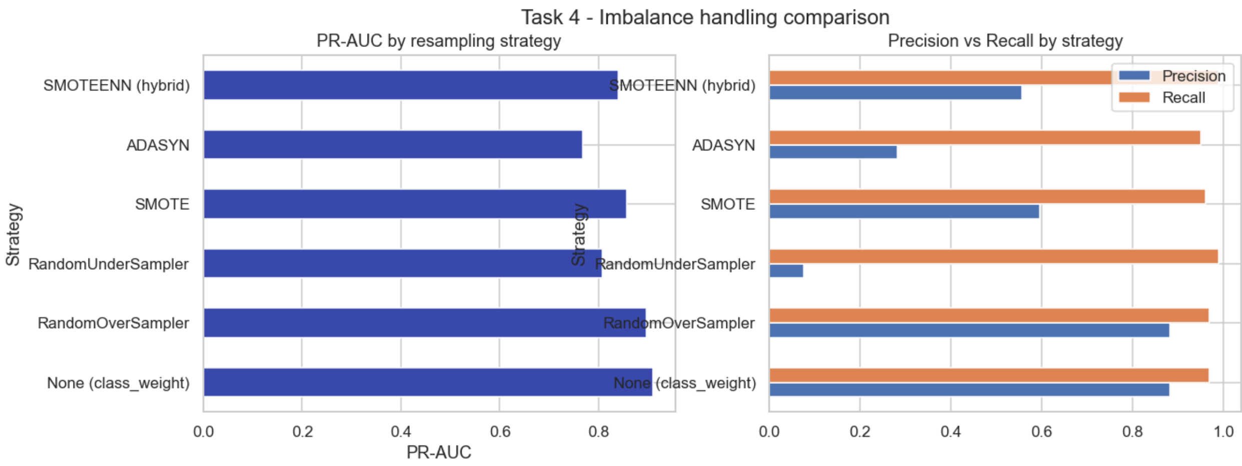


Severe class imbalance



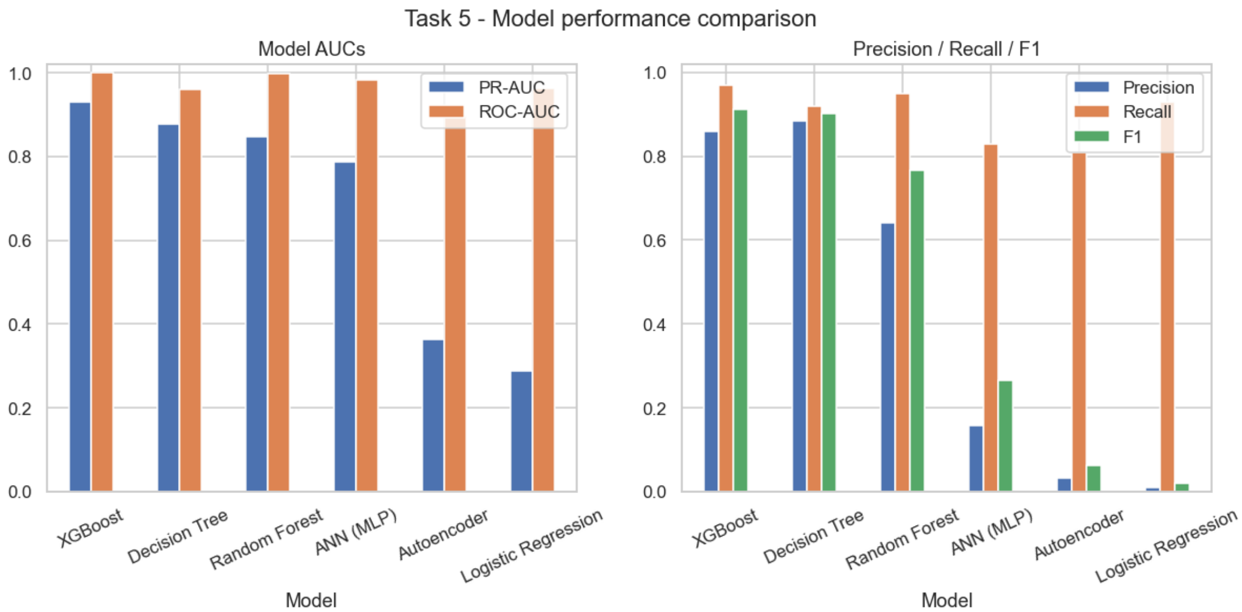
Fraud concentrates in TRANSFER and CASH_OUT

Imbalance handling (Task 4)

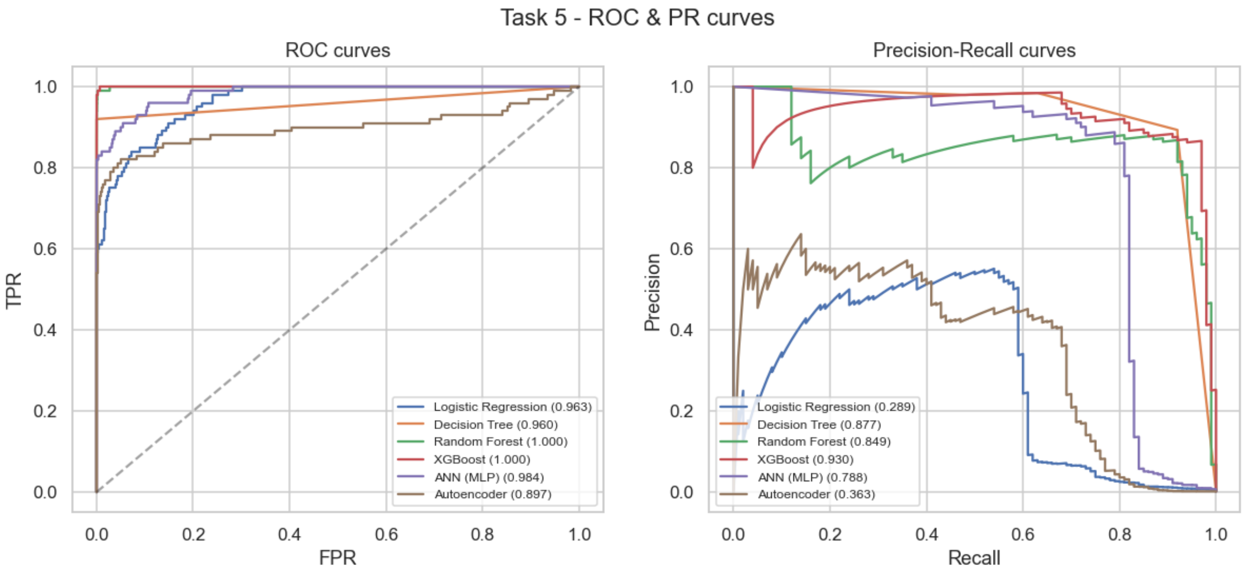


Comparison of resampling strategies

Model performance (Task 5)

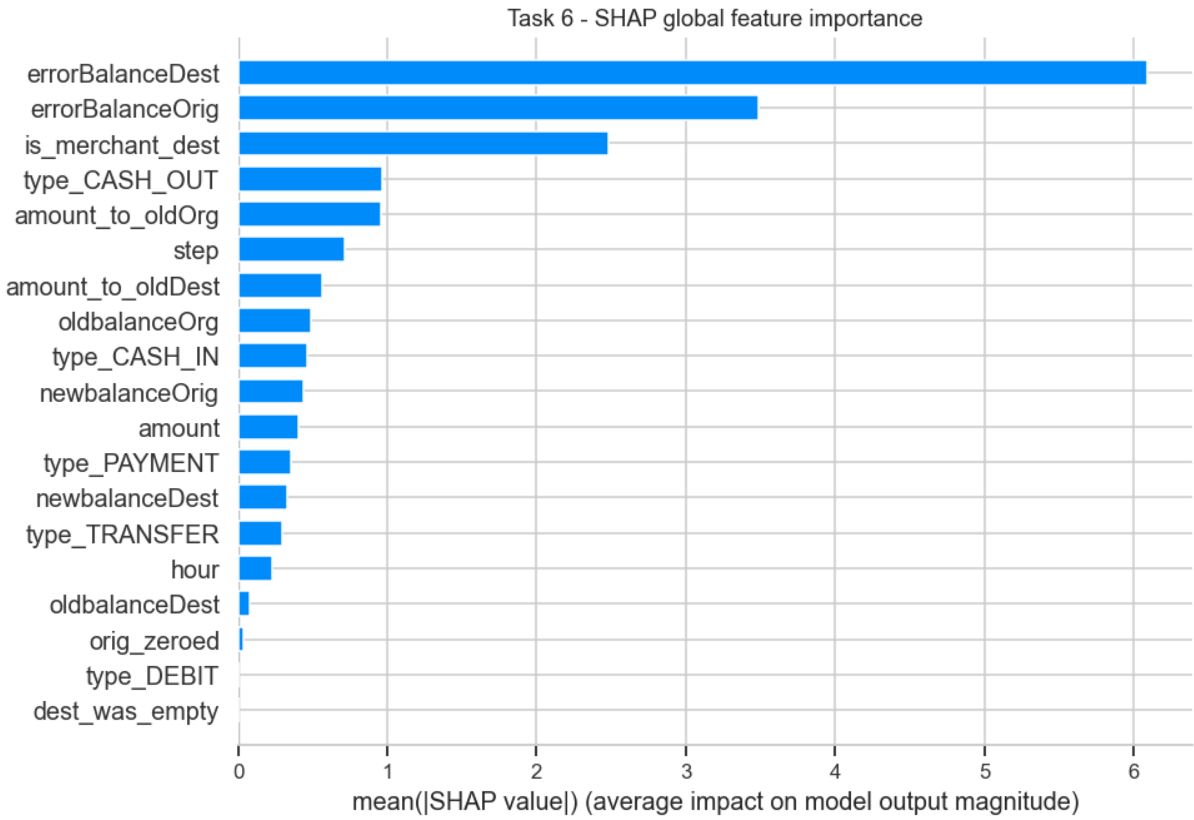


Model performance comparison

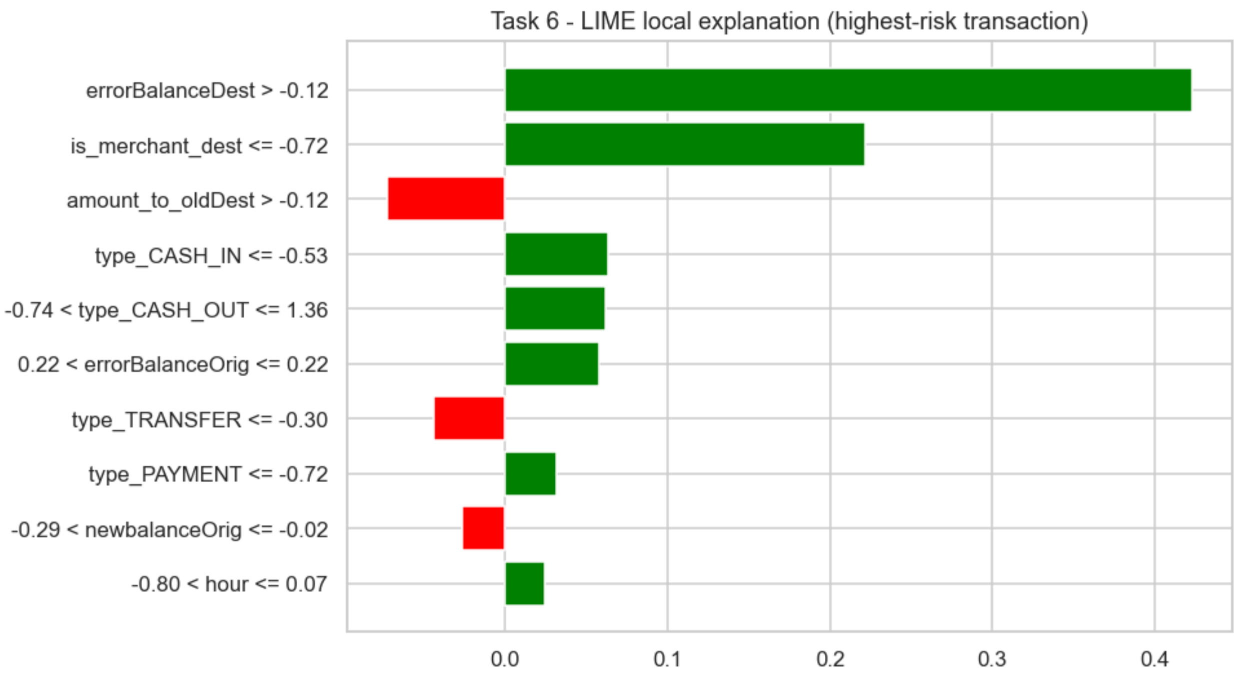


ROC and Precision-Recall curves

Explainability (Task 6)

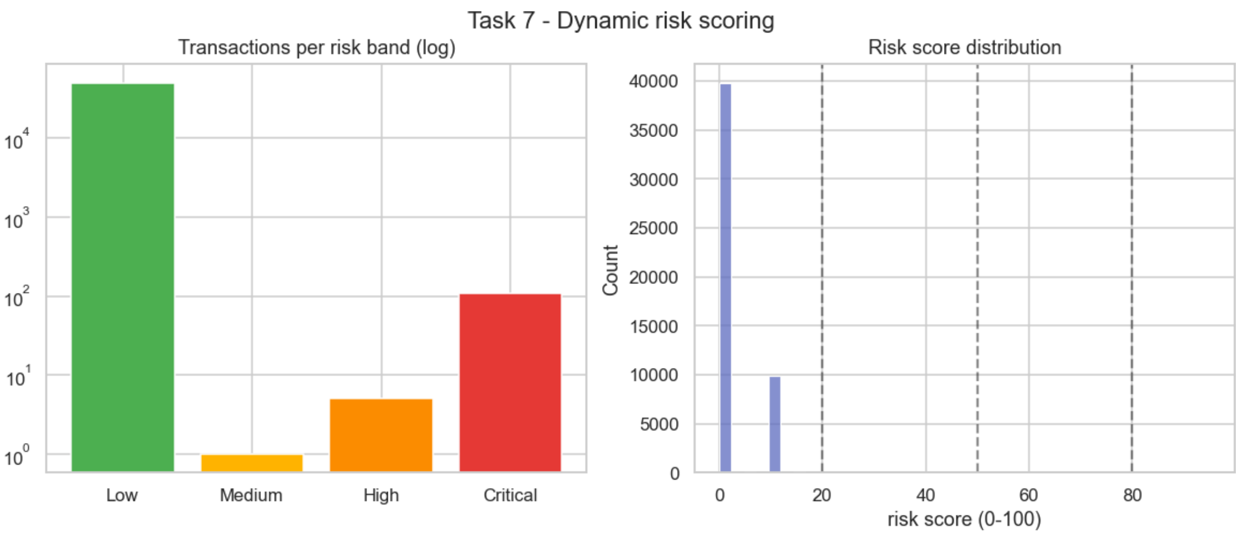


SHAP global feature importance



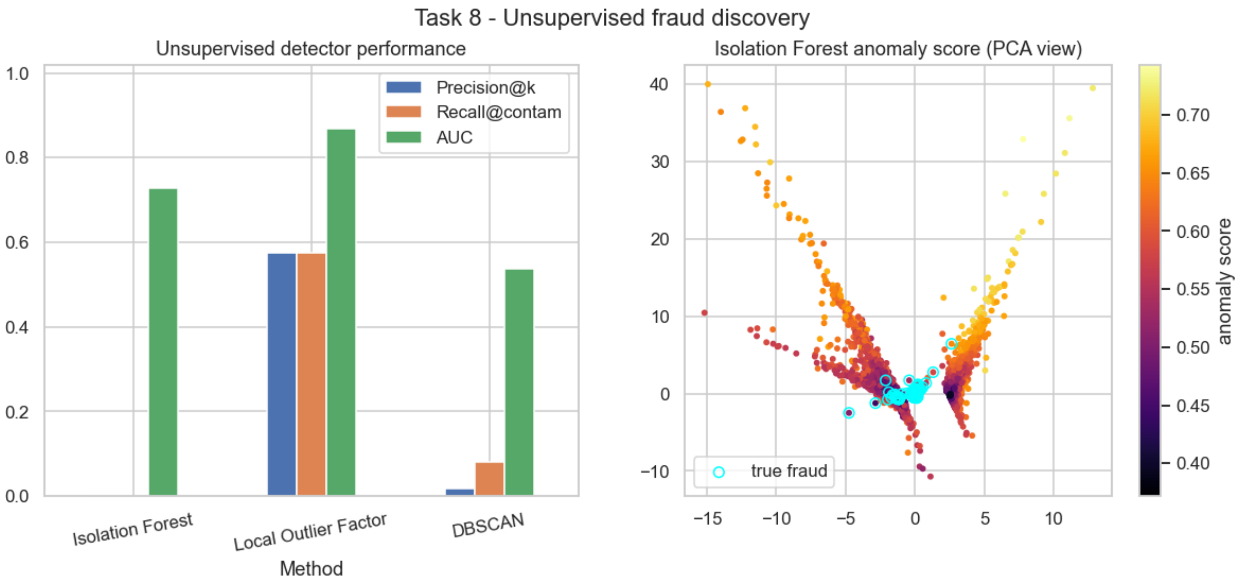
LIME local explanation

Risk scoring (Task 7)



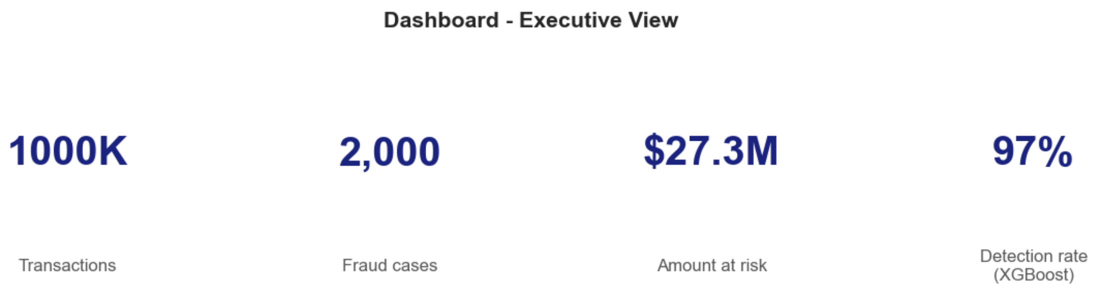
Dynamic risk scoring

Unsupervised discovery (Task 8)

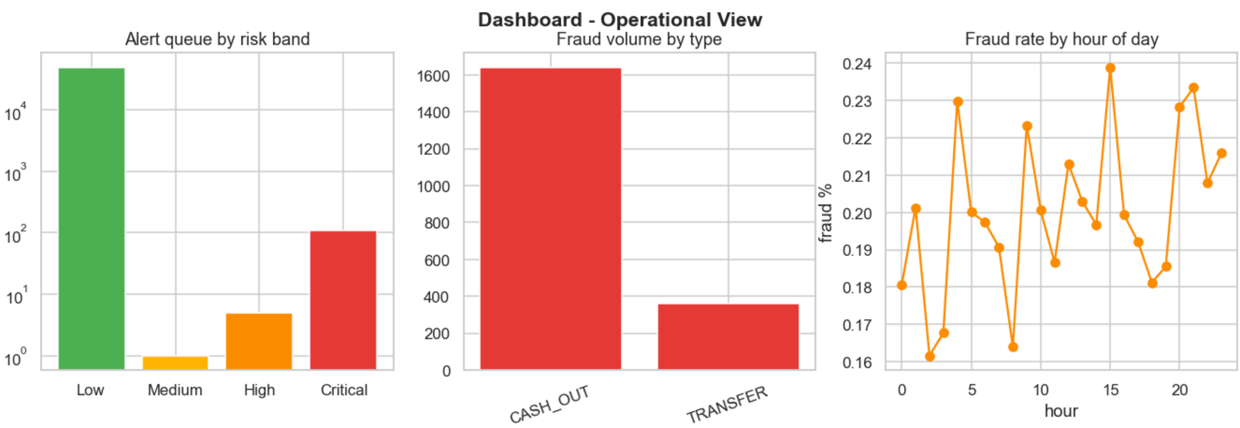


Unsupervised anomaly detection

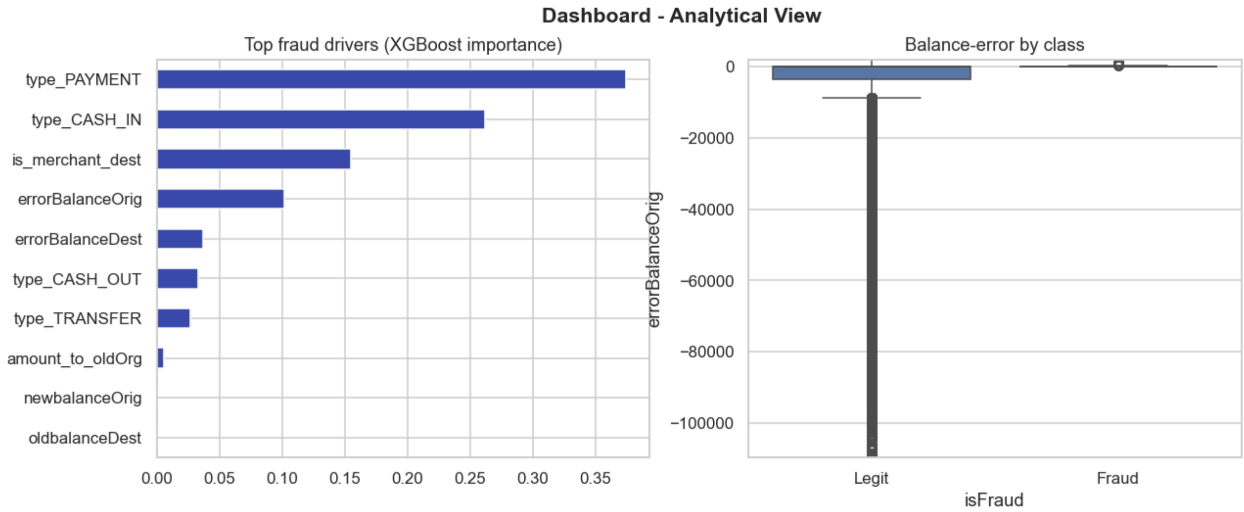
Business-intelligence dashboard (Task 9)



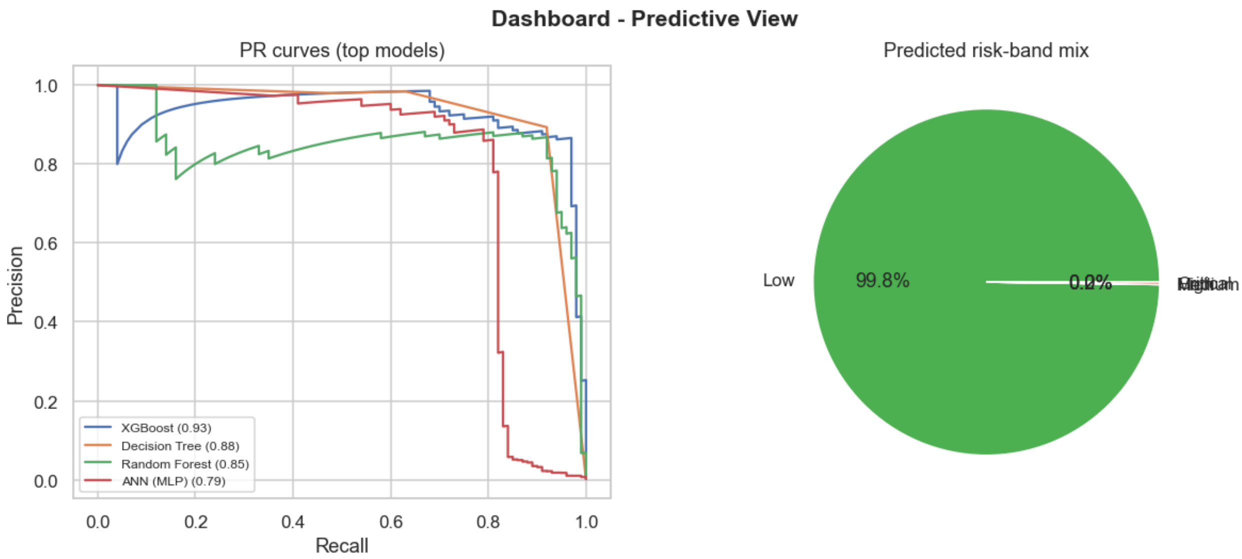
Executive view



Operational view



Analytical view



Predictive view

References

1 Almalki, F., & Masud, M. (2025). *Financial Fraud Detection Using Explainable AI and Stacking Ensemble Methods.* arXiv:2505.10050. https://arxiv.org/abs/2505.10050

2 Baisholan, N., Dietz, J. E., Gnatyuk, S., Turdalyuly, M., & Matson, E. T. (2025). *FraudX AI: An Interpretable Machine Learning Framework for Credit Card Fraud Detection on Imbalanced Datasets.* Computers, 14(4), 120. https://www.mdpi.com/2073-431X/14/4/120

3 Vijayanand, D., & Smrithy, G. S. (2025). *Explainable AI-enhanced ensemble learning for financial fraud detection in mobile money transactions.* Journal of Algorithms & Computational Technology. https://doi.org/10.1177/18724981241289751

4 Cheng, D., Zou, Y., Xiang, S., & Jiang, C. (2024). *Graph Neural Networks for Financial Fraud Detection: A Review.* arXiv:2411.05815. https://arxiv.org/abs/2411.05815